

Cross Language Asymmetric Encryption

Using C# and Javascript

Introduction

Project Context

Developed a

PCI DSS v4.0 Compliant Secure Payment Card Gateway

for a major conglomerate

Technology Stack

- .NET Framework (Microsoft)
- Blazor WebAssembly (Client-side C# rendering)

The Challenge

Requirement:

Encrypt sensitive payment data (card number, CVV, expiry date) on the client side using RSA asymmetric encryption

The Problem

- C# cryptography libraries require operating system access
- OS-level cryptographic APIs are **not available in browser environments**
- Frontend encryption cannot use C#'s native crypto libraries

Solution: JavaScript interoperability with C# for client-side encryption

Key Challenges Faced

Algorithm Compatibility

Ensuring RSA implementation consistency between C# and JavaScript environments

Public Key Formatting

Converting and standardizing key formats across different cryptographic libraries

Data Format Consistency

Maintaining uniform data encoding and padding schemes between platforms

Dual API Integration

Coordinating between .NET and Web Crypto API specifications

RSA Asymmetric Encryption

RSA (Rivest-Shamir-Adleman) is a public-key cryptosystem that uses two different keys:
a public key for encryption and a private key for decryption

Types of Keys in Public Key Cryptography

Public Key

- Shared openly with anyone
- Used to **encrypt data**
- Cannot decrypt data
- Safe to distribute publicly

Private Key

- Kept secret and secure
- Used to **decrypt data**
- Mathematically linked to public key
- Must never be shared

Solution Architecture

JavaScript (Frontend)

1. Receive public key (XML format)
2. Convert XML → JWK format
3. Import key via Web Crypto API
4. Encrypt with RSA-OAEP-SHA256
5. Output Base64 encrypted string



C# (Backend)

1. Generate RSA key pair
2. Export public key as XML
3. Receive encrypted Base64
4. Decrypt with private key
5. Extract original JSON data

Key Format Conversion

Challenge: C# uses XML format, JavaScript Web Crypto API requires JWK format

C# XML Format

```
<RSAKeyValue>
  <Modulus>base64...</Modulus>
  <Exponent>AQAB</Exponent>
</RSAKeyValue>
```



JavaScript JWK Format

```
{
  "kty": "RSA",
  "n": "base64url...",
  "e": "AQAB"
}
```

Conversion Process

1. Parse XML to extract Modulus and Exponent 2. Decode Base64 to byte arrays 3. Convert to Base64URL encoding 4. Construct JWK object for Web Crypto API

XML Format Example

Microsoft .NET Framework RSA Public Key

```
<RSAKeyValue>  
  <Modulus>  
    xG0r3lCd0qz8pAGhN6qGFqVZvGwJrM8K3  
    HZ9x5k8wT9RkQjZoS3bNk2L4xHqpVmY3z...  
  </Modulus>  
  <Exponent>AQAB</Exponent>  
</RSAKeyValue>
```

JWK Format Example

JSON Web Key for Web Crypto API

```
{  
  "kty": "RSA",  
  "n": "xG0r3lCd0qz8pAGhN6qGFqVZvGwJrM8K3HZ9  
    x5k8wT9RkQjZoS3bNk2L4xHqpVmY3z...",  
  "e": "AQAB",  
  "alg": "RSA-OAEP-256",  
  "use": "enc"  
}
```

PEM Format Example

Privacy Enhanced Mail - OpenSSL Standard

```
-----BEGIN PUBLIC KEY-----
```

```
MIIBIjANBgkqhkiG9w0BAQEFAA0CAQ8AMIIBCgKCAQEAXG0r3lCd  
Oqz8pAGhN6qGFqVZvGwJrM8K3HZ9x5k8wT9RkQjZoS3bNk2L4xHq  
pVmY3zQw8K2nR7vXcP4mN9kLsT6wU3eH8jYfG2aB5rC9dE1fK4hI  
mJ0nL8oP6qR2sT4uV5wX7yZ9aB1cD3eF5gH7iJ9kL1mN3oP5qR7s  
T9uV1wX3yZ5aB7cD9eFgHiJkLmNoPqRsTuVwXyZaBcDeFgHiJkLmN  
QIDAQAB
```

```
-----END PUBLIC KEY-----
```

C# Decryption

Using: System.Security.Cryptography (.NET Framework)

RSA Decryption Implementation

```
// Import private key
using (RSA rsa = RSA.Create())
{
    rsa.FromXmlString(privateKeyXml);

    // Decode Base64 to bytes
    byte[] encryptedBytes =
        Convert.FromBase64String(encryptedBase64);

    // Decrypt with RSA-OAEP-SHA256
    byte[] decryptedBytes = rsa.Decrypt(
        encryptedBytes,
        RSAEncryptionPadding.OaepSHA256
    );

    return Encoding.UTF8.GetString(decryptedBytes);
}
```

Real-World Applications

Payment Processing

Encrypt credit card details in browser before transmission

E-commerce, Payment gateways

Healthcare Data

Protect patient information (HIPAA compliance)

Medical portals, Telehealth apps

Authentication Systems

Secure credential exchange during login

SSO, Multi-factor authentication

API Security

Encrypt sensitive API request payloads

Microservices, Third-party integrations

Messaging Platforms

Enable end-to-end encrypted messaging

Chat applications, Email services

Financial Services

Secure banking transactions and account data

Online banking, Trading platforms

Advantages & Benefits

Zero-Knowledge Architecture

Server never sees unencrypted sensitive data

→ Enhanced privacy & compliance

Client-Side Security

Data encrypted before leaving user's device

→ Protection against network sniffing

Language Agnostic

Works across any platform supporting RSA standards

→ Flexibility in tech stack

Industry Standard

Based on proven cryptographic algorithms

→ Trusted & audited

Regulatory Compliance

Meets GDPR, HIPAA, PCI-DSS requirements

→ Legal & business protection

No Third-Party Dependencies

Uses built-in crypto libraries

→ Reduced attack surface

Questions & Answers

Thank You!

Asif Ahmed

□ asifahmed3959.github.io

□ linkedin.com/in/asifahmed3959

